Universidad Carlos III de Madrid

Escuela Politécnica Superior

APUNTES • IA con inspiración biológica.

Aprendizaje evolutivo de la representación de los datos para modelos basados en distancias

Andrea López Salazar

Máster en Ciencia y Tecnología
Curso 2026–2027
7 de noviembre de 2026

1 Problema

Nos piden un programa P que, dado un conjunto de entrenamiento (X_t, y_t), aprenda con técnicas de computación evolutiva una transformación del los atributos: síntesis de atributos o reducción de dimensionalidad. Esa transformación se aplica después al conjunto de validación: $X'_v = v(X_v)$.

Si entrenamos un modelo arbitrario M con los atributos originales y medimos el error absoluto medio (MAE) en validación, obtenemos e. Si entrenamos el mismo m odelo M con los atributos transformados y obtenemos e', el método funciona cuando $e'<e$.

1.1 Planteamiento

- El modelo M es arbitrario. Por eso evaluamos siempre con varios modelos — regresión lineal, KNN y árbol de decisión — y no con uno solo.
- La transformación se aprende solo con el entrenamiento, siendo la validación lo único que usamos para medir.
- Buscamos un método generalizable que funcione igual en datasets distintos sin ajustes a mano.

1.2 Datos

Partimos de dos datasets de regresión: Diabetes (442 muestras, 10 atributos) y California Housing (20640 muestras, 8 atributos). Para comprobar que el método es generalizable, se usan de manera adicional 10 datasets de OpenML — la lista y sus tamaños los tenemos en una sección posterior.

1.3 Protocolo común

- Usamos una partición con 70 % de entrenamiento y 30 % de validación con semillas distintas para cada unas de las repeticiones.
- La métrica va a ser el MAE en el conjunto de la validación.
- Para poder comprobar datasets con escalas muy distintas, usamos el MAE relativo a los datos originales, y el MAE/MAE(crudo), promediado con la media geométrica. Si es menor que 1, significa que hemos mejorado.
- Contrastes estadísticos, incluido el test de Wilcoxon por pares, aplicado sobre el cociente de errores de cada pareja (dataset, partición).

2 Teoría

2.1 Modelos basados en distancias: dependencia de la representación

El KNN predice el valor de un punto nuevo promediando el valor de sus k vecinos más cercanos. La cercanía la defina la distancia, usualmente la euclídea:

$$d(x, x')^2 = \sum_{j=1}^{n} (x_j - x'_j)^2 \tag{1}$$

Esta fórmula trata a todos los atributos por igual, lo que supone un problema porque:

- Si un atributo se mide en miles, y otro en unidades, el primero dominará la distancia, incluso cuando no tiene por qué ser el más informativo.
- Los atributos de menor importancia añaden ruido a la distancia, por lo que muchos vecinos dejan de ser vecinos verdaderos: maldición de la dimensionalidad.
- Con atributos muy sesgados o con valores extremos, unos pocos puntos acaparan prácticamente todo el rango y el resto queda comprimido.

Un árbol de decisión no sufre nada de esto, porque solo compara los atributos con umbrales. Cualquier transformación monótona no cambiará el orden de los datos, y por ende, tampoco el árbol. LO usamos como control: si la transformación aprendida no cambia el error del árbol, es lo que esperamos, y por ende, confirma que la mejora en KNN viene de la geometría y no de otra cosa.

2.2 Transformación aprendida

Para cada atributo j aprendemos dos números, un exponente λ_j y un peso w_j.

2.2.1 Yeo-Johnson

Aquí tenemos en cuenta los exponentes Yeo-Johnson. Es una familia de transformaciones de potencia, que a diferencia de otras como Box-Cox, admite valores negativos y cero.

$$\psi(x;\lambda) = \begin{cases} \dfrac{(x+1)^{\lambda} - 1}{\lambda} & x \geq 0,\ \lambda \neq 0 \\ \log(x+1) & x \geq 0,\ \lambda = 0 \\ -\dfrac{(-x+1)^{2-\lambda} - 1}{2-\lambda} & x < 0,\ \lambda \neq 2 \\ -\log(-x+1) & x < 0,\ \lambda = 2 \end{cases} \tag{2}$$

Con $\lambda = 1$ la transformación es la identidad; con valores menores a 1, comprime los valores grandes — útil en presencia de colas largas a la derecha — y con valores mayores a 1 los expande.

2.2.2 Estandarización y peso

Después de transformar, centramos y escalamos con la media μ_j y la desviación σ_j del entrenamiento y multiplicamos por el peso.

$$z_j = w_j \cdot \frac{\psi(x_j;\lambda_j) - \mu_j}{\sigma_j} \tag{3}$$

Con esto, la distancia euclídea en el espacio transformado es una distancia ponderada en el espacio original:

$$d_\theta(x, x')^2 = \sum_j w_j (\tilde{x_j} - \tilde{x_j^{'}}) \tag{4}$$

Donde $\tilde{x}_j$ es el atributo transformado y estandarizado. El peso w_j^2 es la importancia del atributo j en la distancia. Si $w_j = 0$, el atributo desaparece, y si w_j es grande domina. En términos de aprendizaje de

métricas, es una **métrica diagonal**: matriz de Mahalanobis diagonal, con un paso previo de transformación no lineal por atributo.

2.2.3 Parámetros

El vector que buscamos es $\theta = (\lambda_1, ..., \lambda_n, w_1, ..., w_n)$ con dimensión $2n$ (16 en California y 20 en Diabetes). Lo limitamos a $\lambda_j \in [-2, 2]$ y $w_j \in [0, 3]$.

2.2.4 Punto de partida

Con $\theta_0 = (1, ..., 1)$, la transformación es exactamente la estandarización. Por eso la búsqueda arranca de la línea base "estándar" y cualquier mejora que encuentre es mejora real sobre ella. Esta propiedad la usamos siempre al diseñar los experimentos.

2.2.5 Limitación de λ

Sin límites, el estimador de máxima verosimilitud de Yeo-Johnson dio $\lambda \approx 13,6$ para la longitud de California. Con ese valor la columna se colapsaba prácticamente en un solo valor. Con el rango que se ha definido, eso no ocurre.

2.3 CMA-ES

CMA-ES (Covariance Matrix Adaptation Evolution Strategy) es una estrategia evolutiva para optimización continua sin gradiente. Es la técnica evolutiva que usaremos para buscar θ.

Mantiene una distribución normal multivariante sobre el espacio de búsqueda con media m, tamaño de paso σ y matriz de covarianza C. En cada generación:

1. Muestrea λ_{pop} candidatos: $x_k \sim m + \sigma\mathcal{N}(0, C)$.
2. Evalúa el fitness de cada uno y los ordena.
3. Selecciona los μ mejores y recalcula la media como una media ponderada entre ellos.
4. Adapta el valor de σ - con un camino de evolución que mide si los pasos van todos en la misma dirección o se cancelan - y C - con una actualización de rango uno y otra de rango μ - para que la distribución se alargue en las direcciones que funciona.

Esto nos conviene porque:

1. No necesita gradiente. Nuestro fitness incluye vecinos más cercanos y ajustes locales, que no son derivables.
2. Solo usa el orden de los candidatos, no su valor. Es invariante a transformaciones monótonas del fitness y bastante robusto al ruido.
3. Aprende la forma del problema - qué parámetros están correlacionados o necesitan pasos distintos - con pocos parámetros propios. El tamaño de la población por defecto es: $\lambda_{pop} = 4 + \lfloor 3 \ln D \rfloor$

2.3.1 Implementación

Usamos una implementación propia en la práctica, pero en los experimentos de esta extensión usamos la librería de referencia 'cma' (Hansen), con la que obtenemos resultados equivalentes. Trabajamos en el cubo normalizado $[0, 1]^D$ y reescalamos a los límites de cada parámetro.

2.3.2 Presupuesto

Cada evaluación del fitness es lo más caro — validación cruzada con KNN o con el ajuste local. Usamos 1000 evaluaciones en los experimentos de la extensión y 2020 en el notebook principal. En todas las comparacioens, todos los métodos tienen el mismo presupuesto.

2.4 El fitness. Cómo sabe la búsqueda que un θ es bueno

Para cada candidato θ medimos el error de un modelo sobre el entrenamiento usando validación cruzada de 5 pliegues. Pasos:

1. Subsample de hasta 2000 filas del entrenamiento - para que cada evaluación sea rápida.
2. Transformamos con θ, con la media y la desviación calculadas sobre esas filas.
3. Para cada pliegue ajustamos el modelo y medimos el MAE en el pliegue de prueba.
4. Dividimos entre $s = \overline{|y - \text{mediana}(y)|}$, el error de un predictor trivial que siempre dice la mediana:

$$f(\theta) = \frac{1}{5} \sum_{\text{pliegues}} \frac{MAE_{\text{pliegue}}(\theta)}{s} \tag{5}$$

El fitness es una fracción del error trivial: 0.5 significa que el modelo comeye la mitad de error que un predictor sin información. Esa normalización permite comparar datasets con escalas diferentes. En el notebook principal el fitness promedia regresión lineal y KNN. En la extensión usa el modelo para el que optimizamos: KNN o la regla local lineal.

2.4.1 Riesgo de sobreajuste al fitness

La búsqueda optimiza uan estimación del error, no el error real. Con pocos datos (Diabetes tiene 309 muestras de entrenamiento) la búsqueda aprende el ruido de esos pliegues. Veremos que el fitness mejora y la validación no. Es el problema central que tenemos con el dataset de Diabetes.

2.5 La regla de predicción. De KNN a regresión lineal local.

KNN es una regresión local constante. La predicción es la media de los k vecinos. Funciona bien si la función es plana en el entorno, pero cuando hay pendiente comete un sesgo, sobre todo en los bordes de los datos.

La regresión lineal local ponderada ajusta, para cada punto de consulta q, un hiperplano a sus vecinos, y evalúa en q. Para los k vecinos z_j con valores y_i:

$$min_{\beta_0,\beta} \sum_{i=1}^{k} w_i(y_i - \beta_0 - \beta^T(z_i - q)) + r\overline{w}||\beta|| \tag{6}$$

Y la predicción es $\hat{y}(q) = \beta_0$. Como centramos en q, el término independiente es directamente la predicción.

1. Pesos de vecindad tricúbicos: $w_i = (1 - u_i)$ con $u_i = \frac{d_i}{d_{max}}$ de modo que lso vecinos lejanos pesan menos y el último pesa casi cero.
2. Ridge r: estabiliza el ajuste cuando hay pocos vecinos o atributos muy correlacionados. No penalizamos el término independiente.
3. Parámetros de la regla: k y r. En el método final, los aprende CMA-ES junto a θ.

Es la idea clásica de la *locally weighted regression* Atkenson, Moore y Schaal, 1997; y de la regresión polinómica local de la estadística no paramétrica. Lo que añadimos es optimizar la métrica de la distancia directamente para esta regla, en lugar de reutilizar la que era buena para KNN.

2.6 Qué hay en la literatura y dónde nos situamos

Aprender métricas para el KNN ya es un campo establecido. LMNN y NCA se plantean para la clarificación. Para la regresión, Weinberger y Tesauro aprenden la métrica minimizando el error de regresión por kernel con gradiente, y existe una adaptación de LMNN a regresión con algoritmo evolutivo. También se han publicado artículos donde se ponderan atributos con CMA-ES. Un ancho de banda por variable elegido por validación cruzada es, en la práctica, una métrica diagonal optimizada para el estimador de regresión no paramétrica y elimina de forma automática las variables irrelevantes. Cambiar la regla de predicción manteniendo una métrica fija lo hace DNNR, que usa información diferencial entre los vecinos.

Lo que no encontramos publicado es una combinación concreta de:

- Exponente de Yeo-Johnson y peso por atributo
- Optimización con CMA-ES

- Directamente contra la regla local lineal
- Evaluación de forma sistemática frente a controles y modelos de referencia.

3 Código del método

1. Primero se realiza la configuración y se establece el estilo de las gráficas.
2. Carga de los datos, se descargan también 10 datasets de OpenML. Nos quedamos solo con atributos numéricos y si un dataset tiene más de 10.000 filas, usamos submuestra fija de 10.000 para que el coste sea comparable.
3. `ajustar(xt, theta)` recibe el entrenamiento y un vector $\theta = (\lambda, w)$ y devuelve una función 'v' que transforma cualquier conjunto de datos. Lo importante es que la media y la desviación se calculan solo con el entrenamiento y se reutilizan para la validación.
4. Para las reglas de predicción tenemos KNN y regresión lineal local. Las dos funciones están escritas con NumPy por bloques para que sean rápidas, ya que el fitness las llama miles de veces. `knn_rapido` da el mismo resultado que el `KNeighborsRegressor` (salvo en empates entre distancias iguales) y el `llr` es la regresión lineal local de la que se ha hablado en la teoría.
5. El fitness: cada función `hacer_fitness_*` devuelve una función `f(theta)` que se puede pasar a cualquier optimizador. Los pliegues de validación cruzada se fijan una vez (con la semilla), de modo que todos los candidatos se evalúan sobre los mismos pliegues y la comparación entre ellos es justa.
6. La búsqueda con CMA-ES se realiza con la función `buscar_cma` que envuelve la librería de `cma`. Trabajamos como hemos dicho antes en el cubo $[0, 1]^D$y reescalamos a los límites reales de cada parámetro, guardando el mejor fitness hallado tras cada evaluación para poder dibujar la curva de convergencia. La búsqueda arranca en θ_0 (la estandarización), con un peso inicial de $\sigma_0 = 0.2$ (un 20 % del rango).
7. Elegimos la k y la regularización sin usar la validación. En las configuraciones que no aprenden esos parámetros con CMA-ES (las líneas base), los elegimos por la validación cruzada dentro del entrenamiento, de modo que la comparación sea justa.

4 Demostración

Ejecutamos el método completo sobre una partición de cada dataset de la práctica, con un presupuesto reducido: `PRESUPUESTO_DEMO` evaluaciones, en lugar de las 1.000 de los experimentos, para ver cada paso funcionando. Los números exactos no coincidirán con los del final, ya que promediarán muchas más particiones, pero la tendencia si que la compartirán. En cada dataset se calculan las siguients configuraciones.

Configuración	Qué es
crudo + KNN	e del enunciado: KNN con los atributos originales
estándar + KNN	línea base: atributos estandarizados, k por validación cruzada
estándar + LLR	regla local lineal con los atributos estandarizados
métrica(KNN) + KNN	métrica aprendida con CMA-ES para KNN
métrica(KNN) + LLR	la misma métrica, con la regla local lineal
métrica(LLR) + LLR	método completo: métrica, k y ridge optimizados para la regla local lineal

Tabla 1. Configuraciones evaluadas.

Los resultaods de la aplicación de estas configuraciones los tenemos en la Tabla 2. Cada celda es el cambio del MAE respecto a KNN con los atributos originales (la referencia al error del enunciado). Un valor significa que $e' < e$, o lo que es equivalente, que mejoramos.

Configuración	california	diabetes
crudo + KNN	+0.0 %	+0.0 %
estándar + KNN	−45.8 %	−1.3 %
estándar + LLR	−49.7 %	+1.5 %
métrica(KNN) + KNN	−58.5 %	+5.4 %
métrica(KNN) + LLR	−59.7 %	+1.2 %
métrica(LLR) + LLR	−60.6 %	+3.4 %

Tabla 2. Variación del error respecto a crudo + KNN.

En California vemos que la transformación aprendida mejora mucho respecto a KNN, porque los atributos tienen escalas y colas muy distintas. La regla local lineal también mejora. En Diabetes los cambios son pequeños, y vemos que alguno tiene incluso signo negativo. Hay pocos datos y todos los atributos están ya en una escala parecida o idéntica, por lo que hay poco margen y mucho ruido. Con una sola partición no se puede concluir nada sobre este segundo dataset.

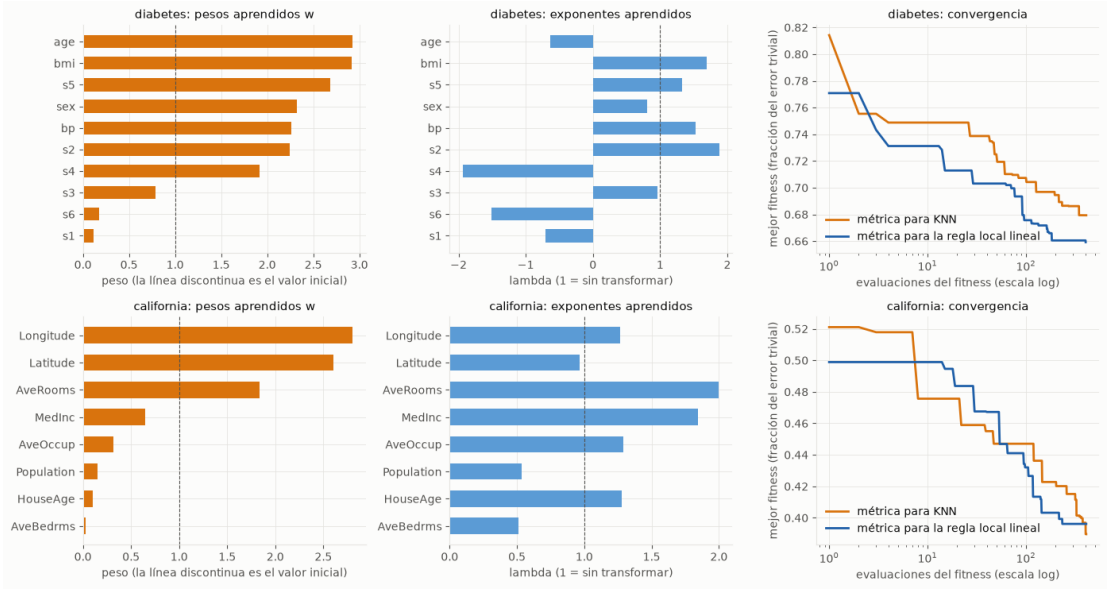


Figura 1. Desglose de componentes para los resultados del KNN.

En la Figura 1 podemos ver en la columna de la izquierda el valor de los pesos resultantes. Un peso

por encima de 1 significa que el atributo cuenta más que en la estandarización. Un peso cercano a 0 prácticamente queda ignorado. En el caso del dataset de California, vemos que tiene sentido que el mayor peso lo tengan la latitud y la longitud, sobre todo dado el dominio del dataset. Usualmente casas con precios elevados suelen encontrarse en zonas o localizaciones parecidas.

En la columna central vemos los exponentes. Un valor lejano a 1 indica que la búsqueda ha decidido deformar el atributo (por ejemplo, comprimir una cola larga). Los atributos con λ cercano a 1 se quedan prácticamente como estaban.

En la columna derecha vemos la convergencia. La curva siempre es no creciente, ya que guarda el mejor valor hasta ese momento. Empieza en el fitness de θ_0, que es el de la línea estándar, por lo que la distancia entre el inicio y el final de la curva es la mejora que logra la búsqueda sobre el entrenamiento. Una curva que sigue bajando al final indica que más presupuesto podría ayudar, y una curva plana puede indicar que la búsqueda empieza a estancarse.

Hay que tener cuidado con Diabetes, ya que una bajada del fitness no garantiza que baje el error de validación. Además, vemos que la bajada es mucho más brusca en California que en Diabetes, lo cual indica que el primer dataset seguramente pudiera obtener una mejora con un presupuesto mayor.

5 Experimentos y resultados

Cada experimento se hace con un script que guarda sus resultados en un CSV. Aquí vamos a leer los resultados y exponerlos, siguiendo el orden en el que los hemos ido haciendo. Dos utilidades que usamos en todas las secciones son: una función para el contraste por pares y otra para leer los datos de los CSV.

5.1 CMA-ES sobre Diabetes y California

El primer experimento es el del notebook prncipal de la práctica. Aprendimos θ con CMA-ES — fitness : media de regresión lineal y KNN con 2020 evaluaciones — en 13 particiones de cada uno de los dos datasets. Aquí recalculamos los errores de lso datos originales — los e del enunciado — y los comparamos con los de CMA-ES. Estos resultados los recogemos en la Tabla 3

Dataset	Modelo	e	e'	Cambio	Mejora en	p (Wilcoxon)
california	KNN	0.826	0.299	−63.8 %	13/13	0.0002
	Regresión lineal	0.533	0.486	−8.7 %	13/13	0.0002
	Árbol de decisión	0.463	0.462	−0.2 %	11/13	0.0134
diabetes	KNN	47.100	48.183	+2.5 %	4/13	0.1677
	Regresión lineal	44.904	45.011	+0.2 %	4/13	0.0681
	Árbol de decisión	61.948	62.510	+0.9 %	7/13	0.8926

Tabla 3. MAE con los atributos originales (e) y con la transformación aprendida (e'), promediado sobre 13 particiones. *Mejora en* indica en cuántas particiones $e' < e$; p es el p-valor del test de Wilcoxon de rangos con signo.

La Figura 2 nos muestra el cambio del MAE al aplicar la transformación aprendida. Cada punto es una partición, y la raya negra es la media. Los puntos por debajo de cero son mejoras. Lo que interesa mirar es la dispersión, y aque si todos los puntos están en el mismo lado del cero, la mejora es consistente. Si están repartidos en ambos lados, la media no significa nada.

En la Figura 2 vemos que en California la mejora es sistemática. Los 13 puntos de KNN están apiñados en torno a un -64 %, y en regresión lineal todos caen alrededor de un 9 %. Por eso el valor de Wilcoxon es tan bajo.

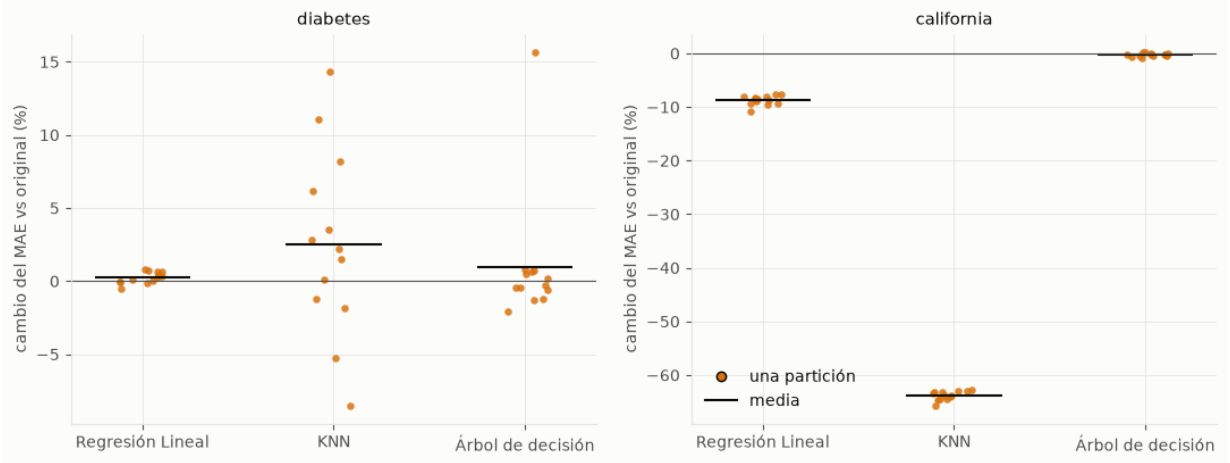


Figura 2. Cambio del MAE al aplicar la transformación aprendida, respecto a los atributos originales, en cada una de las 13 particiones. Cada punto es una partición y la línea negra, la media. Valores negativos indican mejora.

En Diabetes la media de KNN esconde una dispersión enorme que va desde un -8 % a un 14 %. Esto no necesariamente quiere decir que la transformación empeore de forma consistente, sino que es inestable para este dataset. Con tan pocos datos, la métrica aprendida acaba sobreajustándose en unas particiones, mientras que consigue acertar en otras.

Pasando al árbol de decisión en Diabetes, hay un punto en 15 % que tira de la media hacia arriba, el resto se reparte en torno a cero. Es coherente con el valor de p=0.89, porque el test de rangos no se deja arrastrar por un valor extremo tanto como la media. En California el cambio es prácticamente nulo, lo cual tiene sentido dado que los árboles son invariantes a transformaciones monótonas por atributo; una métrica que reescala atributos apenas les afecta.

Con estos resultados nos preguntamos si realmente importa el algorirmo evolutivo, qué ocurre en otros datasets y si se puede mejorar la representación o la regla de predicción.

5.2 Los datasets

Antes de generalizar, tenemos que ver con qué clase de datos estamos trabajando. La Tabla 4 muestra el número de muestras y de atributos de cada dataset (después de habernos quedado solo con los atributos numéricos y de limitar a 10.000 filas).

Dataset	Muestras	Atributos	Origen
diabetes	442	10	práctica
california	20 640	8	práctica
concrete_compressive_strength	1 030	8	OpenML
stock	950	9	OpenML
space_ga	3 107	6	OpenML
sulfur	10 000	6	OpenML
kin8nm	8 192	8	OpenML
bank8FM	8 192	8	OpenML
puma8NH	8 192	8	OpenML
house_8L	10 000	8	OpenML
cpu_small	8 192	12	OpenML
wine_quality	6 497	11	OpenML

Tabla 4. Conjuntos de datos utilizados: los dos de la práctica y diez de regresión adicionales de OpenML.

Como se puede observar, son datasets pequeños o medianos, con entre 6 y 12 atributos numéricos. Los hemos escogido principalmente porque estaban diseñados para tareas de regresión, por su tamaño manejable y por su disponibilidad en OpenML.

5.3 Comparación con otras técnicas bioinspiradas e importancia del algoritmo evolutivo.

En el paper de referencia se clasifican los algoritmos evolutivos en cuatro familias: enjambres, bioinspirados no basados en enjambre, físicos y químicos. Quisimos averiguar si había otra técnica mejor que CMA-ES para este problema.

5.3.1 Diseño del test

- Los mismo 20 y 16 parámetros (θ), el mismo fitness (media de regresión lineal y KNN como en el notebook principal) y el mismo presupuesto de 1.000 evaluaciones para todos.
- 20 algoritmos, casi todos de la primera tabla del paper, usando la librería `mealpy`, un algoritmo genético propio, CMA-ES, una búsqueda aleatoria y el punto inicial sin búsqueda.
- El punto inicial θ_0 se incluye en la población de partida de todos para que ninguno parta con desventaja.
- Dos particiones por dataset - test rápido de carácter exploratorio.

Hay que tener cuidado porque algoritmos como BFO, CSO, CSA y FFA gastan muchas evaluaciones por iteración. Con 1000 evaluaciones, apenas consiguen completar una generación, por lo que sus resultados son básicamente los de elegir el mejor de unos 30 puntos aleatorios. No sacaremos conclusiones sobre ellos. En la Tabla 5 podemos ver el rango medio de cada algoritmo según el fitness alcanzado.

En la Figura 3, la barra es la media del cambio del MAE de validación (promedio de regresión lineal y KNN) en las dos particiones. Las barras hacia la izquierda se pueden interpretar como mejoras. Lo que hay que comparar, no obstante, es la diferencia entre los algoritmos, y hay que mirarla en relación a la que hay entre el punto inicial sin búsqueda, y la búsquea aleatoria (barras azules), que es lo que se consigue "sin esfuerzo".

Podemos ver que en California casi todos los algoritmos consiguen entre un 32 % y un 37 % de mejora. Incluso la búsqueda aleatoria logra un 33 %. CMA-ES es el primero por poco, siendo la distancia entre los mejores algoritmos de 2-3 puntos con solo dos particiones. Por ello esto debe cogerse por pinzas y no es concluyente. Lo que importa es la familia de transformaciones, no el algoritmo que la busca.

Algoritmo	Rango medio
CMA-ES	2.0
GA (propio)	2.5
BBO (biogeografía)	3.5
HS (armonía)	3.5
DE	4.5
ICA (imperialista)	6.5
ACOR	7.5
ABC	7.5

Tabla 5. Rango medio de cada algoritmo según el fitness alcanzado (1 = mejor).

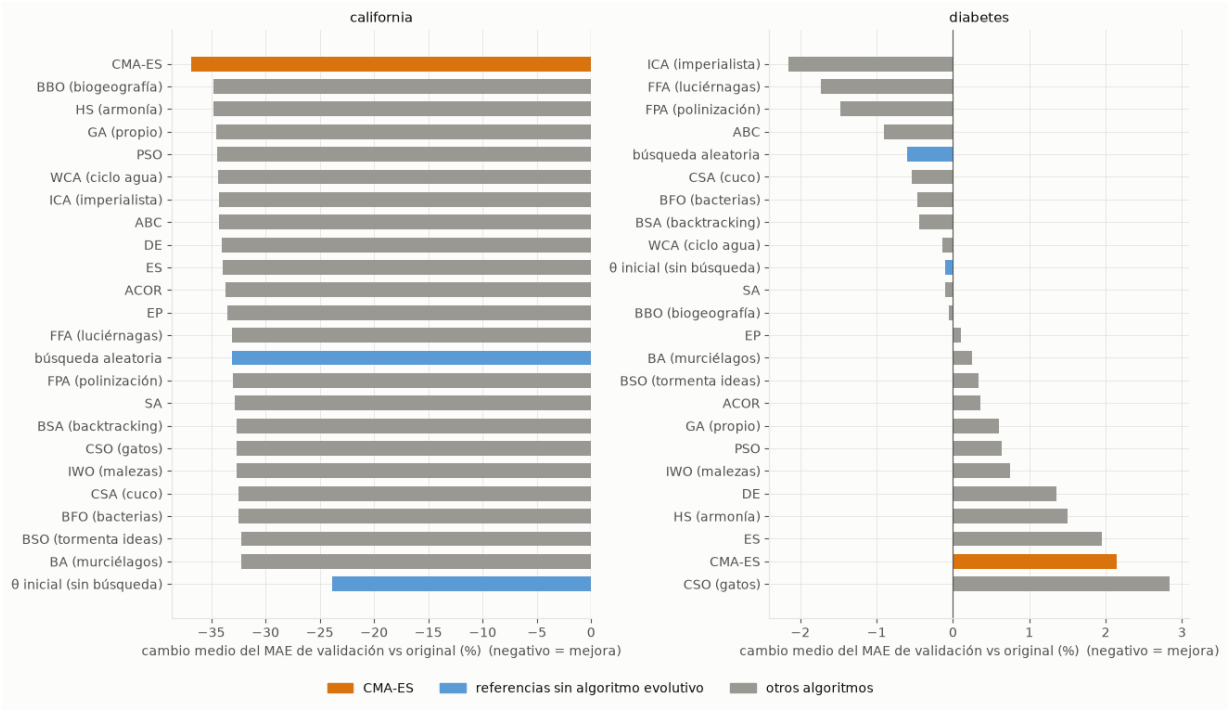


Figura 3. Cambio medio del MAE de validación respecto a los atributos originales para cada algoritmo de optimización. En naranja, CMA-ES; en azul, las referencias sin algoritmo evolutivo (parámetros iniciales y búsqueda aleatoria). Valores negativos indican mejora.

En Diabetes ningún algoritmo mejora de forma fiable, las barras van de -2 % a 3 % y además tienen mucho ruido. CMA-ES sigue tenienod el mejor fitness, pero en validación empeora KNN. El cuello de botella, por ende, no es la potencia de búsqueda.

Nos quedamos con CMA-ES porque tiene el mejor rango medio por fitness (2.0 seguido de cerca por el GA propio con 2.5). Es de los más baratos en tiempo y está justificado teóricamente para este tipo de porblemas continuo, sin gradiente y no separable. No es la única opción razonable, pero es la que se ha escogido.

5.4 ¿Se puede mejorar CMA-ES con un híbrido?

Como Diabetes fallaba por sobreajuste al fitness, probamos a diseñar un algoritmo nuevo que mezclara lo mejor de varios, partiendo del CMA-ES y probando cuatro modificaciones, cada una pensada para un problema concreto. Estas modificaciones quedan descritas en la Tabla 6 Además todas se evalúan con 1.000 evaluaciones y 5 particiones por dataset.

En la Tabla 7 podemos ver la diferenci amedia del cambio del MAE (puntos porcentuales) entre cada una de las variantes respecto al CMA-ES base, sobre 6 particiones. Valores positivos nos indican que la variante ha empeorado sus resultados. La columna ”Mejor que base” cuenta cuántas de las particiones han conseguido mejorar a CMA-ES.

Adicionalmente, la Figura 4 nos muestra de manera visual este cambio medio del MAE. Las barras

Variante	Qué añade	Para qué
CMA+folds	Cambia los pliegues de validación cruzada en cada generación (el fitness pasa a ser estocástico)	Que la búsqueda no se ajuste a unos pliegues concretos
CMA+DE	Inyecta en cada generación un individuo de cruce diferencial (de evolución diferencial)	Más diversidad, algo propio de DE y los algoritmos genéticos
CMA+folds+sel	Lo primero más una selección final: reservamos 100 evaluaciones para puntuar los mejores candidatos en pliegues nuevos, y solo aceptamos el aprendido si mejora a θ_0 en más de 0.005	Elegir de forma robusta
Híbrido completo	Pliegues cambiantes, DE y selección final	Todo junto

Tabla 6. Variantes híbridas de CMA-ES evaluadas.

Dataset	Variante	Dif. media (p.p.)	Mejor que base	p (Wilcoxon)
california	CMA+folds	+7.09	0/6	0.031
	CMA+DE	−0.28	4/6	0.562
	CMA+folds+sel	+0.40	3/6	0.562
	Híbrido (folds+DE+sel)	+0.30	2/6	0.688
diabetes	CMA+folds	+1.24	3/6	0.688
	CMA+DE	+1.53	3/6	0.438
	CMA+folds+sel	+0.98	1/6	0.312
	Híbrido (folds+DE+sel)	+1.57	0/6	0.031

Tabla 7. Diferencia media, en puntos porcentuales de cambio del MAE, entre cada variante y CMA-ES base, sobre 6 particiones. Valores positivos indican que la variante es peor. *Mejor que base* cuenta las particiones en que la variante mejora a CMA-ES.

representan la media en 6 particiones, mientras que la línea vertical sobre ellas representa la desviación típica entre las particiones. Si las líneas de dos variantes se solapan demasiado, la diferencia entre ellas pasa a no ser fiable.

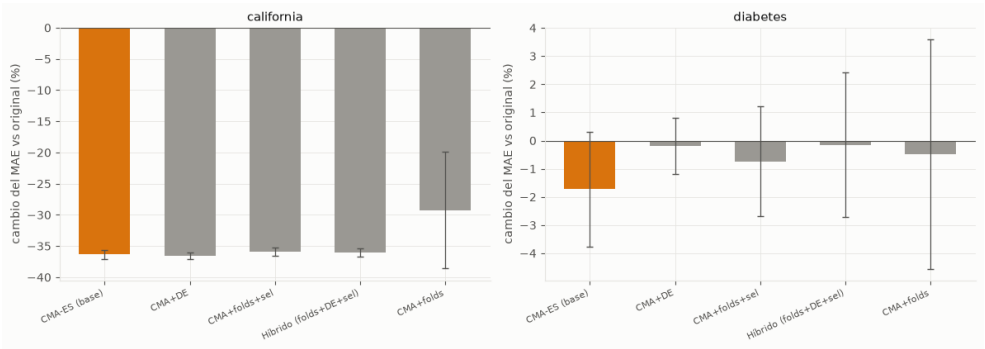


Figura 4. Cambio medio del MAE respecto a los atributos originales para CMA-ES base (en naranja) y sus variantes híbridas, sobre 6 particiones. Las barras de error indican la desviación típica entre particiones. Valores negativos indican mejora.

Vemos que ninguna variante mejora al CMA-ES base. En California, DE y la selección robusta no cambian nada: tenemos diferencias de décimas de punto. En Diabetes ninguna mejora y el híbrido completo es peor en las seis particiones. Además, los pliegues cambiantes empeoran de manera clara en California. Unos siete puntos porcentuales peor, con una dispersión entre las particiones casi trece veces mayor. Esto puede deberse a que en cada generación se están comparando a candidatos con un ruido distinto y CMA-ES — que se apoya en ordenar bien a los candidatos de una generación — avanza peor. Nuestra hipótesis era que este híbrido conseguiría reducir el sobreajuste en Diabetes, pero no hemos podido confirmarlo.

Lo que aprendemos de esta sección es que inventar un algoritmo nuevo no tenía sentido en este punto. Lo que mueve los resultados son la representación y la regla de predicción, no la búsqueda en sí. Por ello, el resto de experimentos los centraremos en esto.

5.5 Introducción de PCA dentro de la búsqueda para la reducción de ruido.

La idea es que la propia búsqueda decida qué componentes principales conservar, como un método alternativo de reducir el ruido. La representación pasaría a ser:

1. Yeo-Johnson
2. Estandarización
3. Pesos por atributo
4. PCA
5. Pesos por componente

Con todos los pesos en 1 es una rotación, y las distancias son idénticas a las de la estandarización, por lo que la búsqueda también arranca de la línea base. El peso de cada componente lo aprende CMA-ES: un peso cercano a 0 equivale a descartar ese componente (reducción de ruido aprendido). Añade n parámetros ($3n$ en total). Probamos también una variante con solo dos parámetros — un corte y una suavidad — y la comparamos con el PCA clásico que conserva el 95 % de la varianza.

El diseño lo hacemos en dos fases para evitar el sesgo de selección. Una primera tanda con semillas del 13 al 15 se usan para elegir la variante, y una tanda de confirmación con 10 particiones nuevas (semillas de 30 a 39) que permite decidir antes de mirar que el PCA solo entraría en el método final si mejoraba de forma consistente. Las siguientes gráficas usan la tanda de confirmación. En la Tabla 8 vemos que la versión con PCA mejora solo en un 0.003, una diferencia no significativa. Esto nos puede indicar que una rotación de los ejes no aporta nada sobre reescalar los atributos originales.

Transformación	KNN
Crudo	1.000
Estándar	0.871
CMA-ES (YJ + pesos)	0.748
CMA-ES + PCA (peso por componente)	0.745

Tabla 8. MAE de KNN con cada transformación, relativo al obtenido con los atributos originales (crudo = 1).

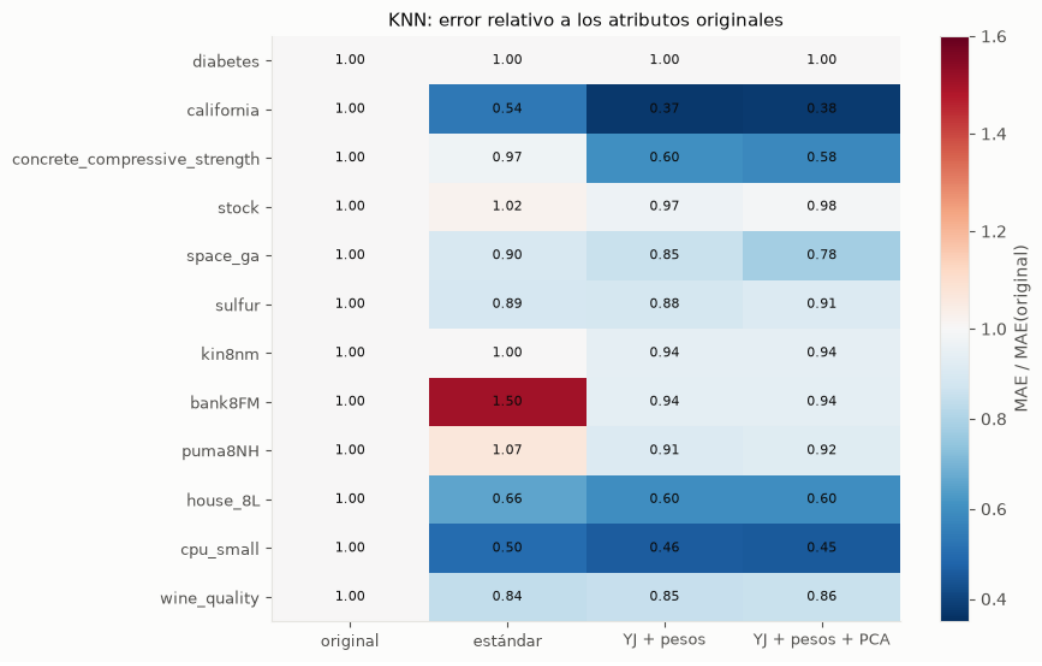


Figura 5. MAE de KNN con cada transformación, relativo al obtenido con los atributos originales, en los 12 datasets. Azul indica mejora (valor menor que 1) y rojo, empeoramiento.

En la Figura 5 vemos un mapa de calor donde cada celda es el cociente entre el error de ese método y el del dataset con los atributos originales. Los valores son, por ende, la media geométrica de las columnas. Por debajo de 1 (azul) se mejora, y por encima del 1 (naranja) empeora. Una columna de azules uniformes nos indica que el método funciona por lo general, mientras que una fila de naranjas indica un dataset difícil.

Se observa que estandarizar solo mejora en unos datasets y empeora en otros — por ejemplo el de banca, donde ya parte de un putno mucho peor que los datos originales. La columna de "YJ + pesos" es mucho más azul que "estándar", mejorando en 11/12 datasets. Además, concluimos que añadir el PCA apenas cambia la columna, ya que se ve casi igual que la de "YJ + pesos". De hecho, en la Figura 6 podemos ver el cambio medio del error al añadir PCA por dataset (barras). Un asterisco indica que ese cambio es estadísticamente distinguible de cero con 10 particiones. Si las barras se reparten a ambos lados, el efecto medio se anula; y esto es lo que observamos en la mayoría de casos.

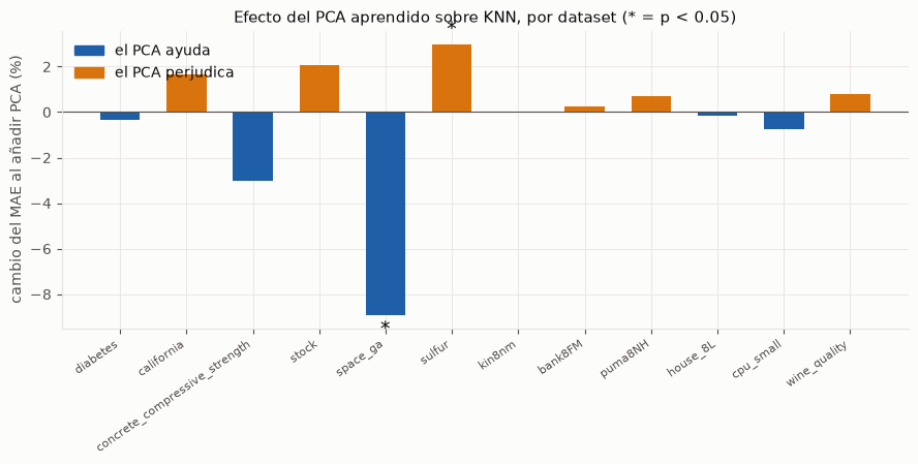


Figura 6. Cambio del MAE de KNN al añadir la rotación PCA a la transformación aprendida (YJ + pesos), por dataset. En azul, los datasets donde PCA mejora; en naranja, donde empeora. El asterisco marca diferencias significativas ($p < 0.05$, test de Wilcoxon).

Se observa que en conjunto, el PCA apenas aporta nada. El cambio global es de apenas un -0.4 % y el p-valor se acerca a 1. Con el criterio fijado, el PCA queda por fuera del método final. Aunque eso no quiere decir que no ayude en todos los datasets, de hecho, aporta bastante en `space_ga` (alrededor de un 9 % menos de error, mejorando en todas las particiones). Esto puede deberse a que los atributos ya estaban de partida muy correlacionados, que es donde rotar puede tener sentido. Sin embargo, perjudica a otros datasets como `sulfur` y `stock`.

El PCA clásico con el corte en el 95 % de varianza empeora la estandarización en la primera tanda — 0.896 frente a 0.873 de media — por lo que quitar componentes de forma fija no parece ser la mejor forma de reducir el ruido.

En la primera tanda de 3 particiones el PCA parecía mejorar en los 12 datasets, pero cuando lo intentamos confirmar con más particiones nuevas la ventaja despaarece. Por eso insistimos en usar particiones que no se usaron para elegir a la hora de evaluar si los cambios en el método son significativos.

5.6 Cambiar la regla de predicción: regresión lineal local.

Como lo que mueve los resultados es la regla de predicción y no el algoritmo evolutivo, vamos a probar con una regla local lineal. El experimento separa dos preguntas con cinco configuraciones: 6 particiones nuevas en cada uno de los 12 datasets. Estas preguntas y configuraciones se recogen en la Tabla 9.

En las cuatro primeras k y el ridge se eligen por validación cruzada en una rejilla de 15 combinaciones. En la última los aprende CMA-ES junto con la métrica. Esa última configuración es el método completo.

En la Tabla 10 podemos ver el MAE de cada configuración en relación al de KNN con los atributos originales. Con esto podemos responder a las tres preguntas de la tabla anterior. En primer lugar, que la regla de por sí ayuda, ya que el paso de KNN a LLR con atributos estandarizados — 0.840 a 0.737 — da casi lo mismo que aprender la métrica para KNN (0.724) sin optimizar nada. Además, con la misma métrica, vemos que cambiar la regla ayuda, pasando de un 0.724 a un 0.673 en meida (geométrica). Finalmente, también comprobamos que ayuda optimizar la métrica para la regla, aunque la ganancia es más pequeña.

Configuración	Qué responde
estándar + KNN y estándar + LLR	¿Ayuda la regla por sí sola, sin aprender nada?
métrica(KNN) + KNN y métrica(KNN) + LLR	Con la misma métrica, ¿ayuda cambiar la regla?
métrica(LLR) + LLR	¿Ayuda optimizar la métrica directamente para la regla?

Tabla 9. Comparaciones planteadas entre configuraciones y la pregunta que responde cada una.

Configuración	MAE relativo
estándar + KNN	0.840
estándar + LLR	0.737
métrica(KNN) + KNN	0.724
métrica(KNN) + LLR	0.673
métrica(LLR) + LLR	**0.641**

Tabla 10. MAE de cada configuración relativo al de KNN con los atributos originales (media geométrica sobre los 12 datasets). Menor es mejor.

En la Figura 7, la gráfica de la izquierda muestra la media geométrica del cociente entre el error de cada método y el de KNN con los atributos originales (a menor valor, mejor). El mapa de color de la derecha desglosa dicho cociente por dataset. En la Tabla 11, *Cambio* es la variación media del error entre los dos métodos que se comparan; *Mejor en* cuenta en cuántos de los 72 casos gana el primero, y p es el p-valor del test de Wilcoxon por pares.

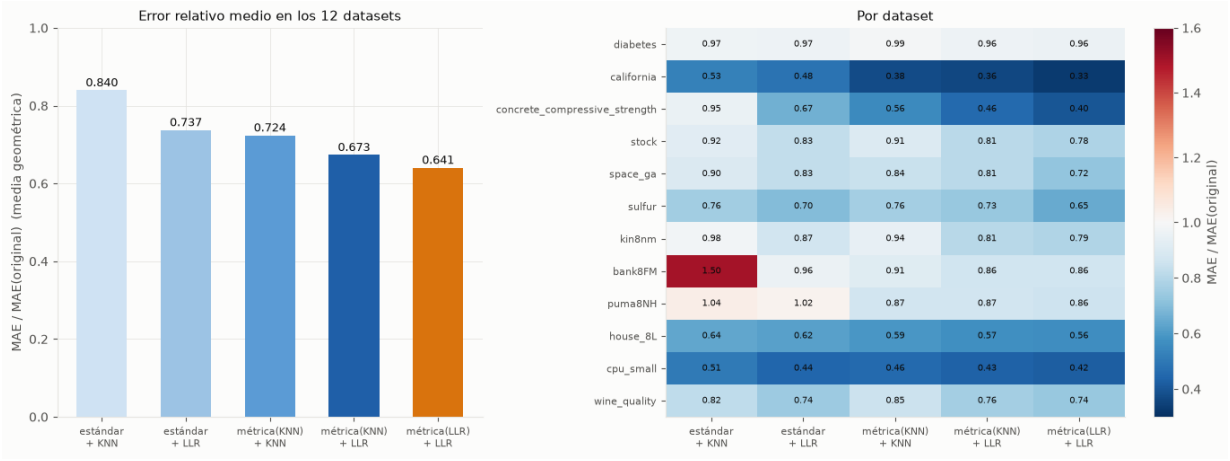


Figura 7. Izquierda: MAE relativo a KNN con los atributos originales, como media geométrica sobre los 12 datasets; en naranja, el método completo. Derecha: el mismo cociente por dataset. Azul indica mejora (valor menor que 1) y rojo, empeoramiento.

Vemos que cada paso mejora al anterior, la media geométrica baja de 0.84 (estándar + KNN) a 0.64 (método completo). Frente a los datos originales, obtenemos un 36 % menos de error. Además, vemos también que la regla local lineal ayuda por sí sola, ya que con datos estándar baja el error un 12 % respecto a KNN, ganando en 68 de 72 parejas. Una parte de la mejora del método completo viene de usar una regla más flexible, y no tanto de la métrica. Frente a la regla local sin aprender, el método completo baja el error un 13 %, y optimizar la métrica para la regla gana casi un 5 % a reutilizar la métrica de KNN con la regla local lineal. Es un cambio pequeño, pero consistente, ganando en 56 de 72 parejas.

Adicionalmente, es consistente entre datasets. El método completo está por debajo de 1 en los 12 datasets. Las mayores mejoras llegan la 60-67 % de reducción del error, siendo el menor `diabetes`, donde solo llega a un 4 %.

Algunas cosas a tener en cuenta son: primero, que la última configuración aprende k y el ridge con 1000 evaluaciones de CMA-ES, mientras que las demás los eligen en una rejilla de 15 combinaciones. Parte de la ventaja puede ser simplemente más ajuste de hiperparámetros. Segundo, tarda mucho más.

Comparación	Cambio	Mejor en	p (Wilcoxon)
estándar + LLR frente a estándar + KNN (la regla sola)	−12.1 %	68/72	< 0.0001
métrica(KNN) + LLR frente a métrica(KNN) + KNN (cambiar la regla)	−6.9 %	67/72	< 0.0001
métrica(LLR) + LLR frente a métrica(KNN) + LLR (optimizar para la regla)	−4.9 %	56/72	< 0.0001
métrica(LLR) + LLR frente a estándar + LLR	−13.2 %	66/72	< 0.0001
métrica(LLR) + LLR frente a estándar + KNN	−23.7 %	69/72	< 0.0001

Tabla 11. Contrastes entre configuraciones sobre 72 casos (12 datasets × 6 particiones). *Cambio* es la variación media del MAE de la primera configuración respecto a la segunda; *Mejor en* cuenta los casos en que la primera tiene menor error.

5.7 ¿Cómo queda frente a otros modelos a nivel absoluto?

Que un método mejore a KNN no dice si es bueno en términos absolutos. Lo comparamos con los modelos que suelen ganar en datos tabulares:

- Random Forest con 200 árboles y Gradient Boosting (`HistGradientBoosting`) ajustados con una búsqueda aleatoria de 30 configuraciones y validación cruzada de 3 pliegues y reentrenados con todo el entrenamiento.
- Proceso gaussiano con ARD — una esclaa por atributo, estimada por máxima verosimilitud. Es el competidor teórico más cercano porque aprende justo lo que aprendemos nosotros, pesos por atributo. Lo entrenamos con una submuestra de 2000 filas por coste, lo que le da menos datos que a nuestro método.

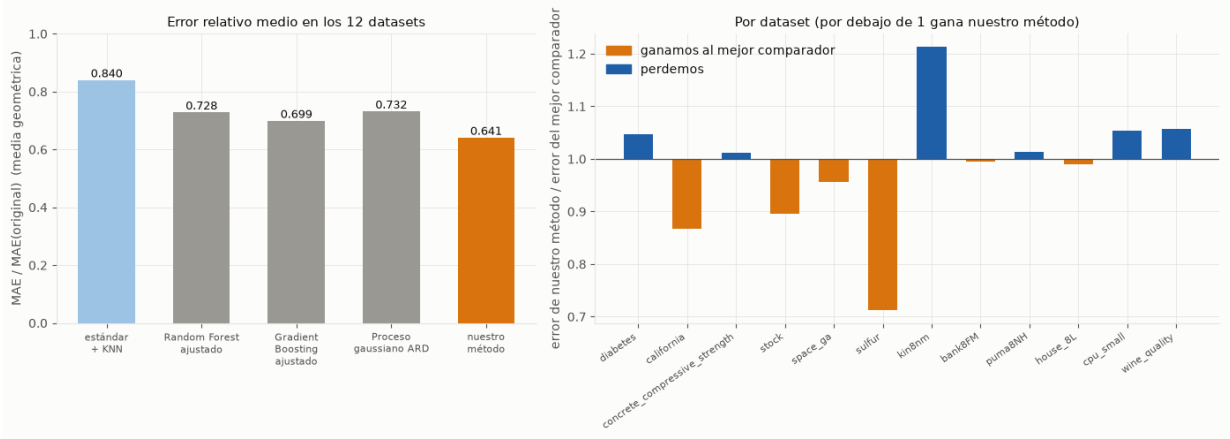


Figura 8. Izquierda: MAE relativo a KNN con los atributos originales (media geométrica sobre los 12 datasets) de nuestro método frente a otros modelos de regresión. Derecha: cociente entre el error de nuestro método y el del mejor de los comparadores en cada dataset; por debajo de 1, nuestro método gana.

En la Figura 8, a la izquierda nos fijamos en los errores relativos, a menor valor, mejor. Y a la derecha, cada barra representa el cociente entre el error de nuestro método y el del mejor de los tres comparadores del dataset: por debajo de 1 (naranja) ganamos, y por encima (azul), perdemos. Este cociente es una comparación dura, porque el mejor comparador se elige después de ver los resultados…

Vemos que de media nuestro método tiene el menor error, y frente a cada uno por separado la ventaja es del 8 % al 12 % y significativa (p de Wilcoxon). Además, frente al mejor de los tres en cada pareja, la diferencia es de aproximadamente un -2 %, lo cual en este caso no es significativo. Es decir, que estamos a la altura del mejor modelo de referencia, pero no por encima. Finalmente, por dataset hay ganadores distintos, por lo que el resultado también depende en gran medida de la naturaleza de los datos y atributos del problema.

En la Tabla 12 podemos ver los contrastes entre nuestro modelo y los otros sobre los 72 casos. La última fila

Comparación	Cambio	Mejor en	p (Wilcoxon)
Nuestro método frente a Random Forest ajustado	−12.0 %	58/72	< 0.0001
Nuestro método frente a Gradient Boosting ajustado	−8.3 %	53/72	< 0.0001
Nuestro método frente a proceso gaussiano ARD	−12.4 %	51/72	< 0.0001
Nuestro método frente al mejor de los tres en cada dataset	−2.1 %	33/72	0.618

Tabla 12. Contrastes entre nuestro método y otros modelos de regresión sobre 72 casos (12 datasets × 6 particiones). *Cambio* es la variación media del MAE de nuestro método respecto al comparador; *Mejor en* cuenta los casos en que nuestro método tiene menor error.

es la más interesante, ya que muestra que ganamos en menos de la mitad de los casos, pero con un cambio medio negativo de -2.1 %. Esto es interesante, porque cuando ganamso lo hacemos por más margen que cuando perdemos. No obstante, con un $p = 0.618$ no hay diferencia significativa. Volvemos a la conclusión de que nuestro método está a la par del mejor modelo elegido a posteriori en cada dataset, y supera de forma significativa a cada uno de ellos por separado.

Sin embargo, otro punto a tener en cuenta es que el proceso gaussiano solo ve 2000 filas, lo que perjudica en los datasets grandes. Nuestro método ajusta más hiperparámetros (exponentes, peso, k, ridge) que Random Forest y Gradient Boosting, que solo ajustan 30 configuraciones aleatorias.

5.8 Caso de Diabetes

Diabetes es el único dataset de la práctica donde el método no cumple el criterio de $e' < e$, y necesitamos saber por qué. Mostramos el cambio del error por partición para los cuatro métodos de la confirmación en 10 particiones.

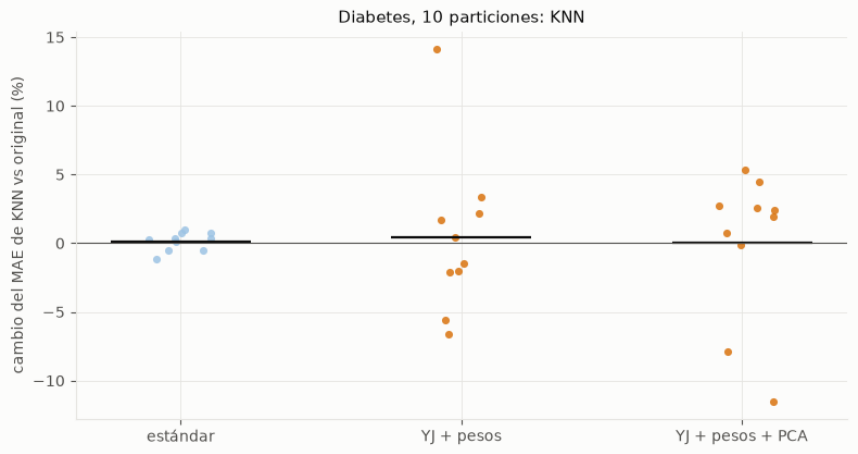


Figura 9. Cambio del MAE de KNN respecto a los atributos originales en diabetes, para cada una de las 10 particiones. Cada punto es una partición y la línea negra, la media.

Transformación	Media	Desviación típica
Estándar	+0.13 %	0.66
CMA-ES (YJ + pesos)	+0.40 %	5.80
CMA-ES + PCA (peso por componente)	+0.07 %	5.45

Tabla 13. Media y desviación típica, sobre 10 particiones, del cambio del MAE de KNN respecto a los atributos originales en diabetes.

En la Figura 9 vemos que los puntos están repartidos a ambos lados de cero, por lo que no podemos demostrar que haya una mejora. En la Tabla 13 se recogen la media y la desviación típica sorbe las 10 particiones, respecto al cambio del MAE de KNN respecto a los atributos originales en diabetes.

Con 10 particiones nuevas, el cambio medio es prácticamente nulo para todos los métodos, con una desviación de unos 5-6 puntos porcentuales entre particiones. Por ende, la conclusión es que no hay mejora en Diabetes. Esto se puede deber a la poca presencia de datos, con solo 309 muestras de entrenamiento para ajustar 20 parámetros con un fitness que usa validación cruzada sobre esas mismas muestras. La búsqueda mejora el fitness — bajada de 5-8 % — pero esa mejora no llega a trasladarse a validación: sobreajuste al criterio de búsqueda…

Probamos regularizar hacia θ_0, usar un único exponente compartido, pliegues cambiantes y selección robusta con validación independiente; pero ninguna mejora Diabetes de forma consistente. Los propios datos explican que el margen sea bajo, ya que los datos ya vienen centralizados y escalas parecidas, por lo que estandarizar no cambia nada y tampoco hay colas largas que requieran de corrección.

En una tanda con solo 2 particiones, CMA-ES sale con +2.2 % en Diabetes, y con 6 particiones con -1.7 % (en ambos casos, promedio de regresión lineal y KNN).El signo cambiaba con el número de particiones. Con una desviación de 5-6 puntos. Por ende, podemos concluir que cualquier interpretación sobre Diabetes con pocas particiones se basa en ruido.

6 Análisis global

6.1 Qué hemos aprendido en orden

En la Tabla 14 se resumen las respuestas a las preguntas que se han ido formulando durante el desarrollo del trabajo.

Pregunta	Respuesta	Evidencia
¿Funciona aprender la transformación con CMA-ES?	Sí, y mucho en California; no en Diabetes	5.0 (13 particiones)
¿Importa qué algoritmo evolutivo se use?	Poco; CMA-ES es la opción más razonable	5.2
¿Mejora un híbrido a CMA-ES (pliegues cambiantes, DE, selección)?	No	5.3
¿Ayuda el PCA dentro de la búsqueda?	No de forma consistente (sí en `space_ga`, no en `sulfur`)	5.4
¿Generaliza a otros datasets?	Sí: KNN mejora un 25 % de media (11 de 12 datasets)	5.4
¿Ayuda una regla local lineal y optimizar la métrica para ella?	Sí: otro 13 % frente a esa regla sin aprender	5.5
¿Es competitivo con modelos de referencia?	Sí, a la altura del mejor de ellos, no por encima	5.6
¿Y en Diabetes?	No mejora; sobreajuste por pocos datos	5.7

Tabla 14. Resumen de las preguntas planteadas y de la sección donde se encuentra la evidencia de cada respuesta.

6.2 Qué podemos afirmar

- Una transformaciín por atributo — exponente de Yeo-Johnson y peso — aprendida con CMA-ES mejora de forma sustancial a todos los modelos basados en distancias. Con 10 particiones nuevas y 12 datasets, KNN reduce su error un 25 % respecto a los atributos originales, y 14 % respecto a la estandarización simple. Con regresión linela, la mejora es del 6-7 % y con el árbol de decisión nula, como se espera por su invarianza.
- La mejora cumple el criterio del enunciado de $e' < e$ en la gran mayoría de datasets para KNN y en la mayoría para regresión lineal. Sin embargo, no llega a cumplirlo en Diabetes.
- Cambiar la regla de predicción a regresión lineal local y optimizar la métrica para ella da una mejora adicional respecto a usar la métrica de KNN. Es un efecto pequeño pero consistente.
- El método es competitivo con Random Forest, Gradient Boosting y procesos gaussianos ARD ajustados. Su error medio es menor que el de cada uno por separado, y comparable al del mejor de ellos en cada caso.

6.3 Lo que no podemos afirmar

- Aprender métricas para KNN, ponderar atributos con CMA-ES y la regresión lineal local están publicados. Lo que no hemos encontrado publicado es la combinación concreta, pero nuestra búsqueda bibliográfica puede estar limitada.
- Tampoco podemos confirmar que el algoritmo evolutivo sea lo que produce la mejora. La búsqueda aleatoria con el mimso presupuesto ya consigue la mayor parte en California. Lo que importa es la familia de transformaciones y la regla de predicción.
- No creemos que supere a modelos de referencia, como mucho estará a su altura con ganadores distintos dependiendo del dataset en el que se aplique.
- Con Diabetes tenemos la hipótesis de que este método no necesariamente tendrá buenos resultados con pocos datos.

6.4 Limitaciones

En primer lugar, los resultados se basan en doce datasets pequeños o medianos, con atributos numéricos, que han sido elegidos por nosotros. No hay datos con muchas dimensiones, atributos categóricos, valores perdidos o muy ruidosos. Además, se han hecho entre 2 y 13 particiones dependiendo del experimento. Las conclusiones sobre Diabetes necesitan muchas particiones por el ruido y las primeras tandas — 2 a 3 particiones — solo son de carácter exploratorio. Por eso confirmamos lo importante con nuevas particiones.

Nuestro método ajusta más parámetros que modelos de referencia, por lo que parte de la ventaja puede venir de ahí. Además, la búsqueda tarda entre 1-3 minutos por partición, po lo que para un movelo que se entrena una vez es asumible, pero no para un uso interactivo. Finalmente, el fitness depende mucho del modelo objetivo. Optimizamos para KNN o para la regla local lineal, por lo que una trnaformación buena para ellos puede perjudicar a otro modelo.